



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



Course Name - API-driven Cloud Native Solutions

Course Number - CC ZG506 / SE ZG506

Units - 5

Instructor Profile



Dr. Shreyas Rao

- 20+ Years of Experience in IT, Teaching and Research
- Working as Associate Professor (Off Campus), Dept. of CSIS, BITS-Pilani, WILP
- B.E from VTU, M.S in Software Systems from BITS (WILP) and PhD from MAHE
- Worked as Business Analyst and Team Lead at SLK Software Services for 7+ years
- Previously worked in Presidency University and Sahyadri College, Mangaluru as R&D Head, CSE
- Executed 10+ Consultancy projects
- COE member in AI&ML and COE member in Data Science (Govt. Sponsored for 1.2 Cr)

Instructor Profile



Consultancy

- ISRO-SAC (Ahmedabad) funded research project titled “Ontology Enabled Disaster Management Web Service using Data Integration” as Technical Consultant. Deployed in ISRO.

Collaboration with Dept. of Health Innovation, Kasturba Hospital, MAHE

- Telemedicine effectiveness during Covid Wave-I at Kasturba Hospital, Manipal
- Study on psychological implications of COVID-19 on Nursing professionals
- Covid prediction using Patient Discharge Data

*Published papers can be viewed at <https://scholar.google.com/citations?user=MFNrrlcAAAAJ&hl=en&oi=ao>

Instructor Profile



Application Development

- Design and Development of AI enabled tool for juvenile self-transformation (Mental Health domain, App Development, Deep Learning & NLP) for Dept. of Psychology, Montfort College.
- SEEC application for MAHE University (Applied patent)
- Grievance Management Portal
- Designed and Developed 'Dhriti', a mental health resource **Chabot** that caters to mental health needs of people during Covid, from the COE in AI&ML, SCEM. Bot is released in Dakshina Kannada region of Karnataka which answers user queries in English, Kannada and Hindi languages. Deployed on the Web and Facebook Messenger channels, used by 5000+ users.

Instructor Profile



Courses I teach at BITS

- API-Driven Cloud Native Solutions
- Design of Conversational Experiences
- Scalable Services
- DevOps for Cloud
- Introduction to Data Science
- Software Engineering
- Introduction to DevOps
- Web Technologies
- Software Architecture

Student Profile



- Name
- Role in Organization (Ex: Developer, Tester, Architect etc.)
- Years of Experience
- Exposure to Cloud Native development? Yes / No (If yes, mention Cloud name)
- Exposure to API-based development? Yes / No



BITS Pilani
Pilani Campus



Walkthrough of Course Handout

Course Description



Analyze, Design, Develop and Deploy **cloud native applications** in innovative areas such as **Data Science, Artificial Intelligence/Machine Learning (AI/ML), IoT, Data Analytics** etc.; Build an end to end complex application; Extensive usage of well-known **PaaS/APIs**; Demonstration of compliance with relevant, industry adapted best practices; Deployment using modern strategies; Presentation of the Milestones and outcomes in appropriate forms; Periodic review of progress of the project by faculty.



Course Objectives



No	Objective
CO1	Explain the fundamentals of APIs, including RESTful and GraphQL APIs, and demonstrate how to design, specify, and implement API versioning
CO2	Identify the key components of the cloud-native ecosystem such as DevOps, microservices, containers, and serverless computing
CO3	Apply APIs and relevant tools to execute end-to-end data science and machine learning projects
CO4	Design AI assistants using APIs for use cases in natural language processing, speech recognition, and computer vision, and construct generative AI assistants using the RAG approach based on suitable language models
CO5	Develop and deploy end-to-end IoT and data analytics projects using APIs, serverless functions, and containers



Module Structure



Module	Description
0	Python Programming [Self-Study]
1	API Basics
2	Cloud Native Application
3	Data Science and Machine Learning
4	AI and Cognitive Services Part A: Basic APIs for AI applications Part B: Language Models Part C: Cloud Native Application Deployment
5	IoT and Data Analytics



Contact Session wise Coverage

Session	Coverage
CS01	API Basics (OpenAPI, REST, gRPC, GraphQL)
CS02	API Basics + Cloud Native Application
CS03	Cloud Native Application
CS04	Introduction to Data Science & Machine Learning and Lifecycles
CS05	API driven Cloud-native Data Pipeline
CS06	API-driven ML pipelines
CS07	Tools review; Case Study
CS08	Basic APIs for AI Applications - NLP, CV, Speech recognition
CS09	Language Models (LLMs)
CS10	RAG Models
CS11	RAG Models
CS12	Cloud Native Application Deployment
CS13	IoT and Data Analytics
CS14	APIs for IoT and Data Analytics
CS15	Serverless Functions; Case study
CS16	Course review and discussion

Experiential Learning



Lab. No	Module No.	Session No.	Experiment	Tools list (Indicative only)
1	3	5	Demonstrate the design and development of API-based data pipeline which involves data ingestion, pre-processing, analysis and monitoring activities	Prefect, AirFlow
2	3	7	Demonstrate the design and development of Machine Learning pipeline which involves model development, training, deployment and monitoring activities	AWS SageMaker, MLFlow
3	4	8	Demonstrate NLP tasks such as speech processing, language translation, text summarization, and text generation using Cloud-based APIs	HuggingFace APIs and / or AWS APIs or Azure APIs or GCP APIs
4	4	11	Demonstrate a generative AI assistant, using RAG approach, which answers queries about an application (chatbot)	https://flowiseai.com/ LangChain LangSmith LangFuse Python code to invoke APIs
5	5	14 or 15	Demonstrate API-based IoT and Data Analytics application	OpenRemote ThingsBoard

Evaluation



Evaluation Component	Name	Type	Weight	Duration
EC - 1	Quiz I	Online	5%	
	Assignment I	Online	15%	
	Assignment II	Online	15%	
EC - 2	Mid-term Exam	Closed book	30%	2 hours
EC - 3	End Semester Exam	Open book	35%	2.5 hours

Information!



1. All the sessions will be conducted on MS Teams, in online mode.
2. EC-1 components (Assignments) will be administered on the Taxila portal.
3. All the Course Material (Handout, PPTs etc.) will be uploaded on MS Teams.
4. Announcements will be made on Taxila portal when,
 - (a) Assignment(s) are administered
 - (b) Rescheduling or Cancellation of class (if any)
 - (c) To notify the Syllabus for Mid-Sem and Comprehensive exams
 - (d) To upload the Scheme and Solution documents for Mid-sem and Comprehensive exams.
5. Email for correspondence - shreyas.rao@pilani.bits-pilani.ac.in
[Request - Mention the course code / course name in the subject line]



Lecture No. 1

Agenda



API Basics

- Introduction to APIs
- HTTP APIs
- OpenAPI Specification
- REST, GraphQL and gRPC based APIs
- API Versioning, Semantic versioning
- Examples of API - Google Maps API, Rapid API, Conference API etc.



BITS Pilani
Pilani Campus



API

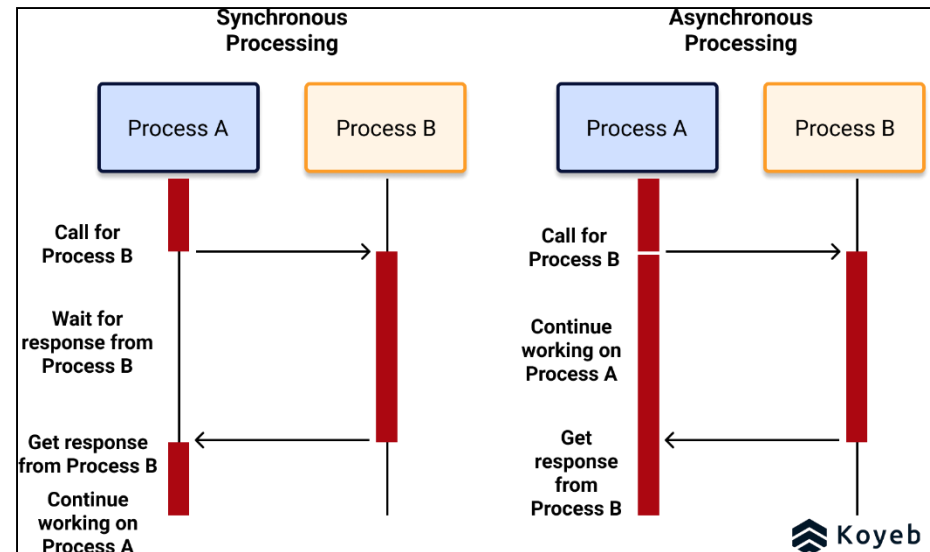
API

- Acronym for “Application Programming Interface”
- Some definitions
 1. API is a contract between a service and its clients
 2. Set of rules and protocols for building and interacting with software applications. The application will be exposed as set of APIs [\[API-first approach\]](#)
 3. APIs enable different software systems to communicate with each other, allowing for data exchange and functional integration *without requiring the end-user to understand the underlying code.*



Synchronous vs Asynchronous

- **Synchronous** execution means the first task in a program must finish processing before moving on to executing the next task [blocked]
- **Asynchronous** execution means a second task can begin executing in parallel, without waiting for an earlier task to finish [non-blocked]



Synchronous APIs

- A synchronous API is one where each request must wait for the previous one to complete before the next one can start.

Ex: REST, gRPC, GraphQL

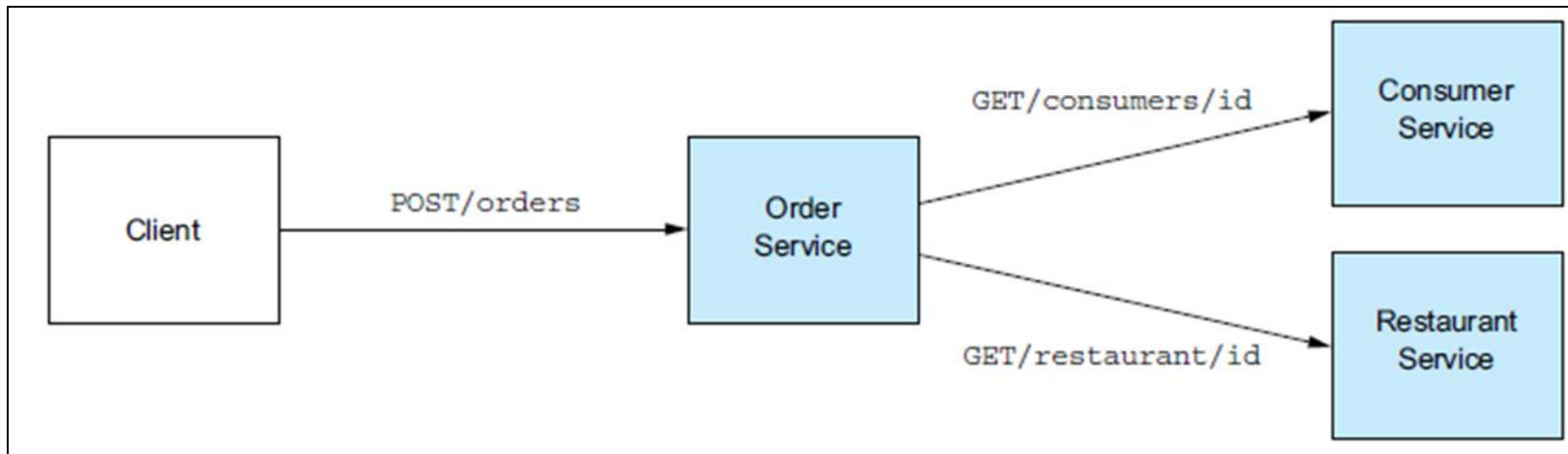


Fig. Order service invokes other services using REST APIs

Asynchronous APIs

- In contrast, an asynchronous API allows multiple operations to be initiated and potentially completed **concurrently**, without waiting for each operation to finish before starting the next one.

Ex: Message Brokers like RabbitMQ, Apache Kafka, SQS etc.

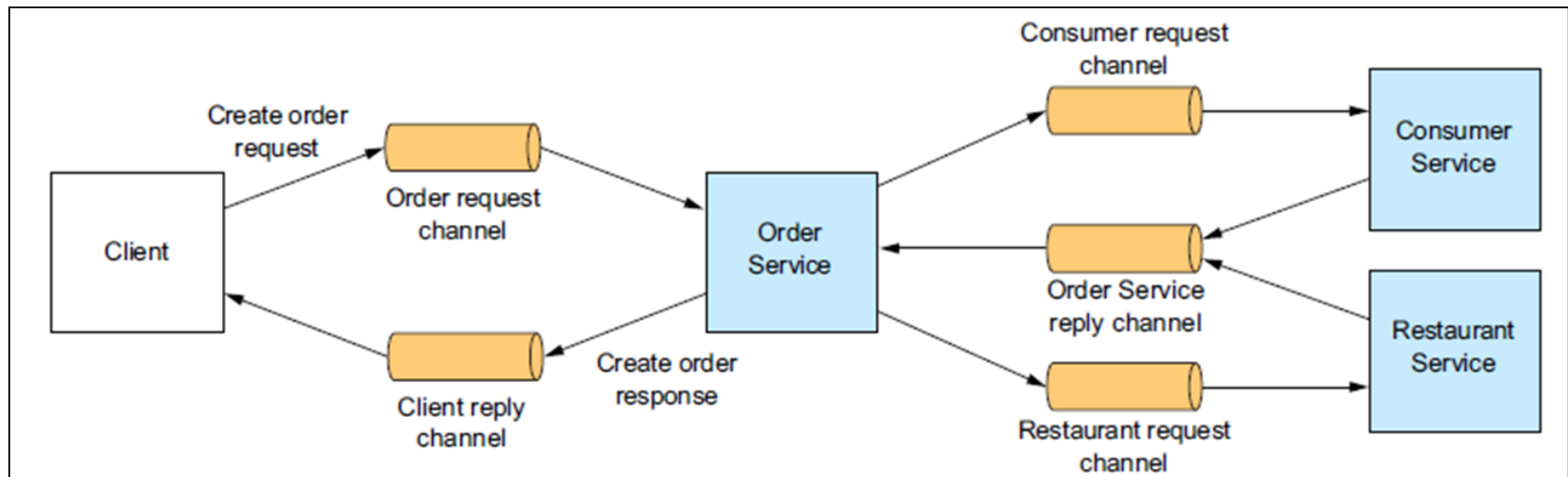


Fig. Client and Order service invokes other services using message brokers



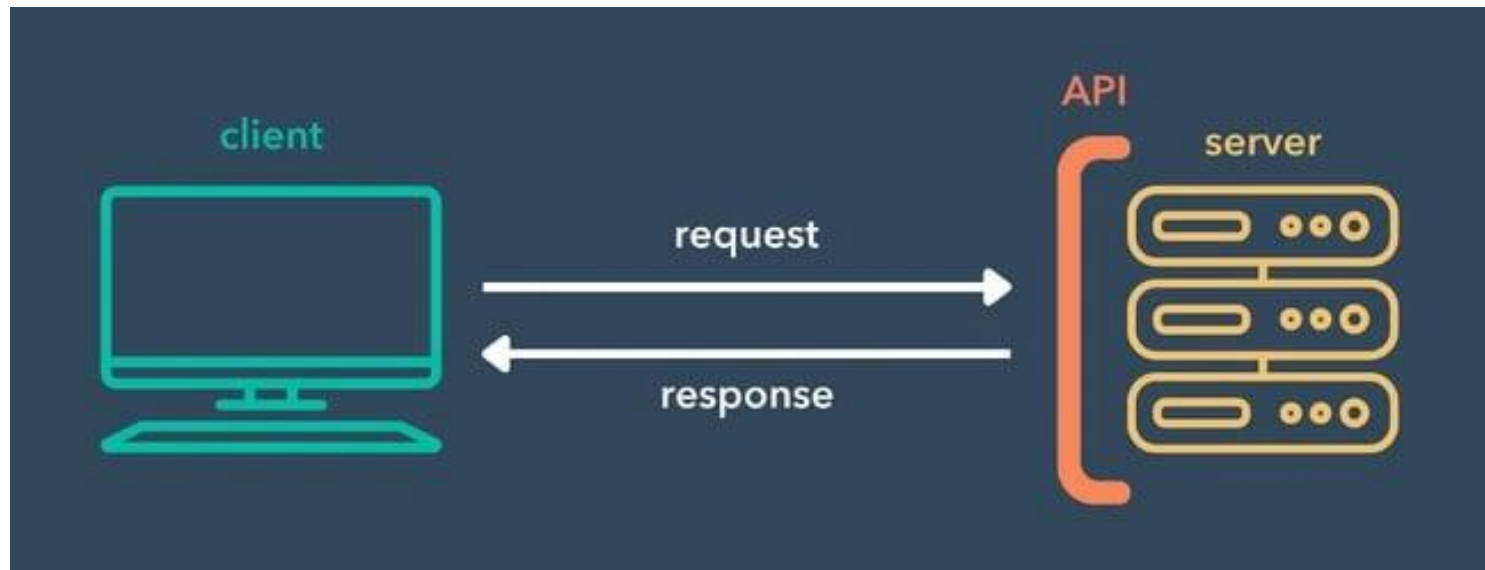
HTTP APIs

HTTP APIs

- HTTP (Hypertext Transfer Protocol) APIs are a standard way for applications to communicate over the web.
- Use HTTP Protocol
- Define a set of rules and conventions for sending and receiving requests and responses between clients (such as web browsers or mobile apps) and servers.
- Typically seen in “Browser -> Server” communication

Components of HTTP API

1. API Endpoints
2. API Request
3. API Response



API Endpoints

- The API endpoints are simple URLs that represent the collection of objects or a single object.
- The resources are found usually on the server, the API endpoints are used to send the request to resources.
- Each endpoint is a URL designed to perform a single function by accessing the required resources.
- Endpoints act as the “doors” or “paths” through which a client application can send HTTP requests to the API server.

API Request

- Client (Browser or Mobile) sends HTTP Request to a specific API endpoint
- Purpose is to request data or perform a specific operation
- Every API request begins with selecting appropriate HTTP method (or verb)

HTTP Methods / Verbs

GET: To retrieve data

POST: To submit data to the server

PUT: To update existing data

DELETE: To delete data

Ex: Facebook or Youtube

API Response

- Refers to the data or information sent back by the API server after processing a request.
- Formats: JSON / XML

HTTP Status Codes



200 OK: The request was successful, and the server is returning the requested data.
201 Created: The request has resulted in creating a new resource on the server.
400 Bad request: The client's request is malformed or contains errors.
401 Unauthorized: Authentication credentials are missing or invalid.
403 Forbidden: The client is not allowed to access the requested resource.
404 Not found: The requested resource does not exist.
500 Internal server error: An unexpected server error occurred.

Try it!



- Open your terminal

```
> curl -X GET "https://jsonplaceholder.typicode.com/posts"
```

HTTP Request - GET

HTTP Endpoint - <https://jsonplaceholder.typicode.com/posts>

HTTP Response - Posts in JSON format

- Try with Fiddler application / Postman



Open API Specification

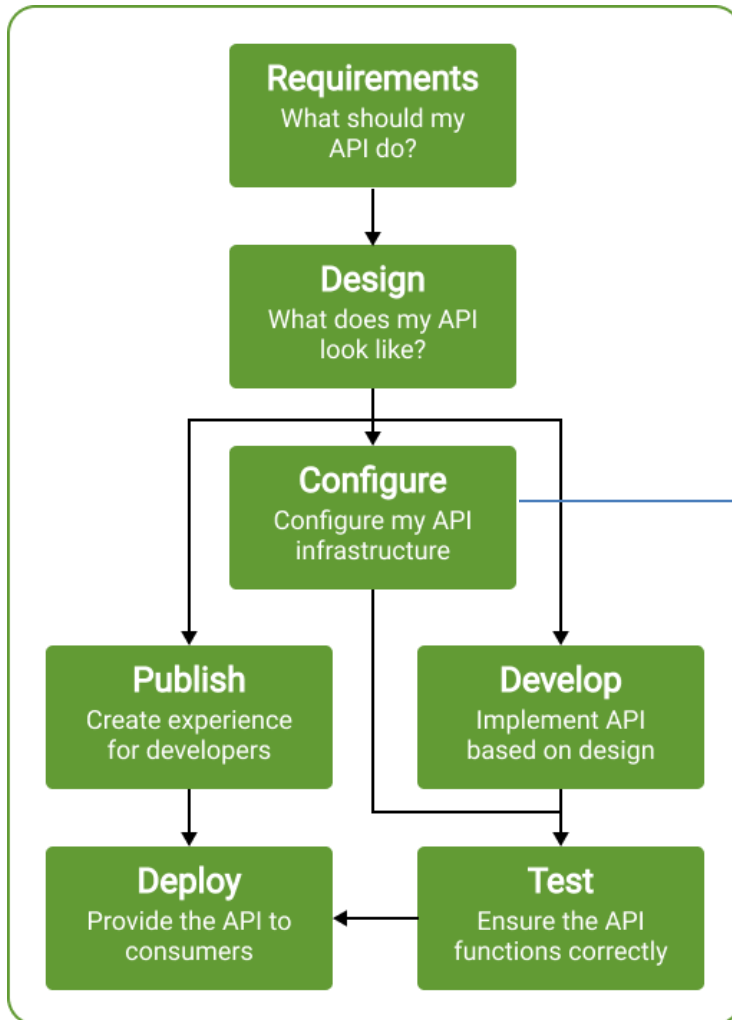
<https://www.openapis.org/>



What is OpenAPI

- API Description Standard
- The OpenAPI Specifications [previously Swagger specification] provides a formal standard for describing HTTP APIs, mainly RESTful APIs.
- This allows people to understand how an API works, how a sequence of APIs work together, generate client code, create tests, apply design standards etc.

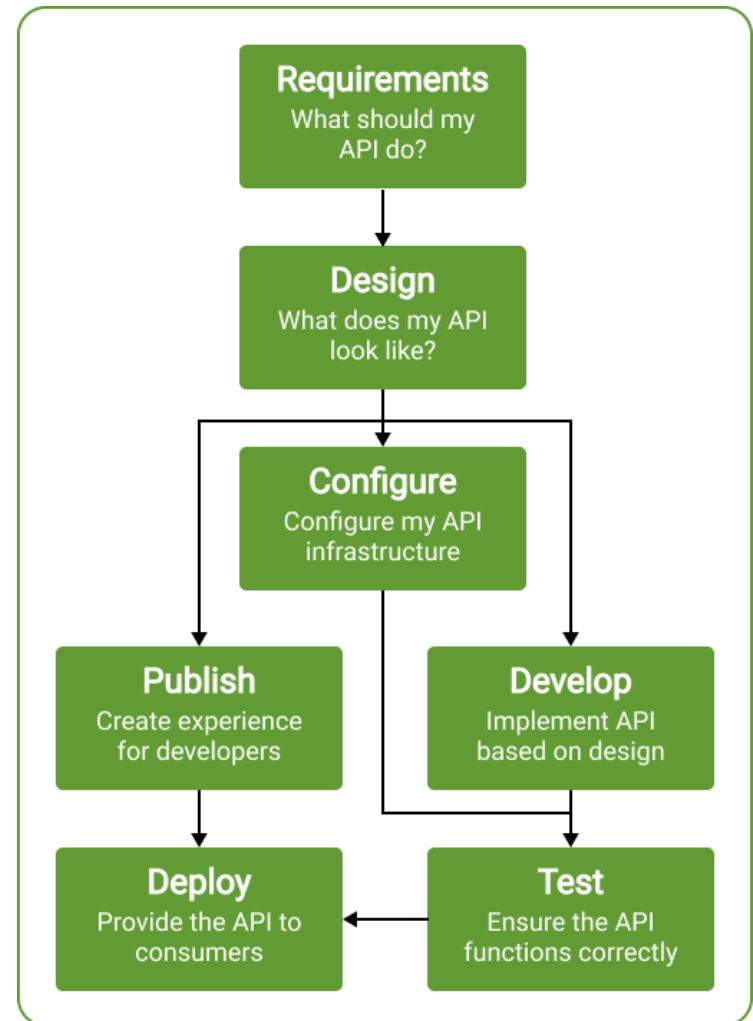
What is OpenAPI



Framework, Server,
Storage, Cloud etc.

Bookstore Scenario

- Let's go through the entire process with a practical example of building a HTTP API for an online bookstore using Python and FastAPI.



1. Requirements

- The API should allow users to manage books [CRUD operations]
 - add a new book
 - retrieve book details
 - update book information
 - delete books

2. Design

Identify the API Endpoints

- GET /books
- POST /books
- GET /books/{id}
- PUT /books/{id}
- DELETE /books/{id}

Sample Data Model for “book”

```
{  
  "id": integer,  
  "title": "string",  
  "author": "string",  
  "isbn": "string",  
  "publishedDate": "string",  
  "price": integer  
}
```

3. Configure and 4. Publish

Configure / Identify:

- FastAPI framework in Python for development
- Uvicorn as the Web Server
- JSON for storing the Book details

Publish the API

- Documentation is auto-generated by FastAPI and available at `/docs`
- URL - Localhost:8000/docs

5. Develop 6. Test 7. Deploy

Develop

- Implement the API using FastAPI.
- Implement all HTTP methods for GET, POST, PUT and DELETE

Test

- Use “pytest” or “unittest” frameworks for testing

Deploy

- Various platforms such as Heroku, AWS, Google Cloud can be used



BITS Pilani
Pilani Campus



REST

REST

- Architectural Style, Architecture of the Web. [Roy Fielding in 2000]
- REST Architecture treats every content as a resource
- Each resource identified by a URI (Uniform Resource Identifier)
- Ex: Web pages, Image, Video, PDF, Dynamic business data etc.
- Resource representation formats - JSON / XML
- HTTP methods for CRUD (create, read, update, delete) operations on resources:
 - **GET**: Retrieve representation of resource
 - **POST**: Create a new resource
 - **PUT**: Update/modify existing resource
 - **DELETE**: Delete an existing resource
- * Suitable permissions are required for POST / PUT / DELETE operations



REST

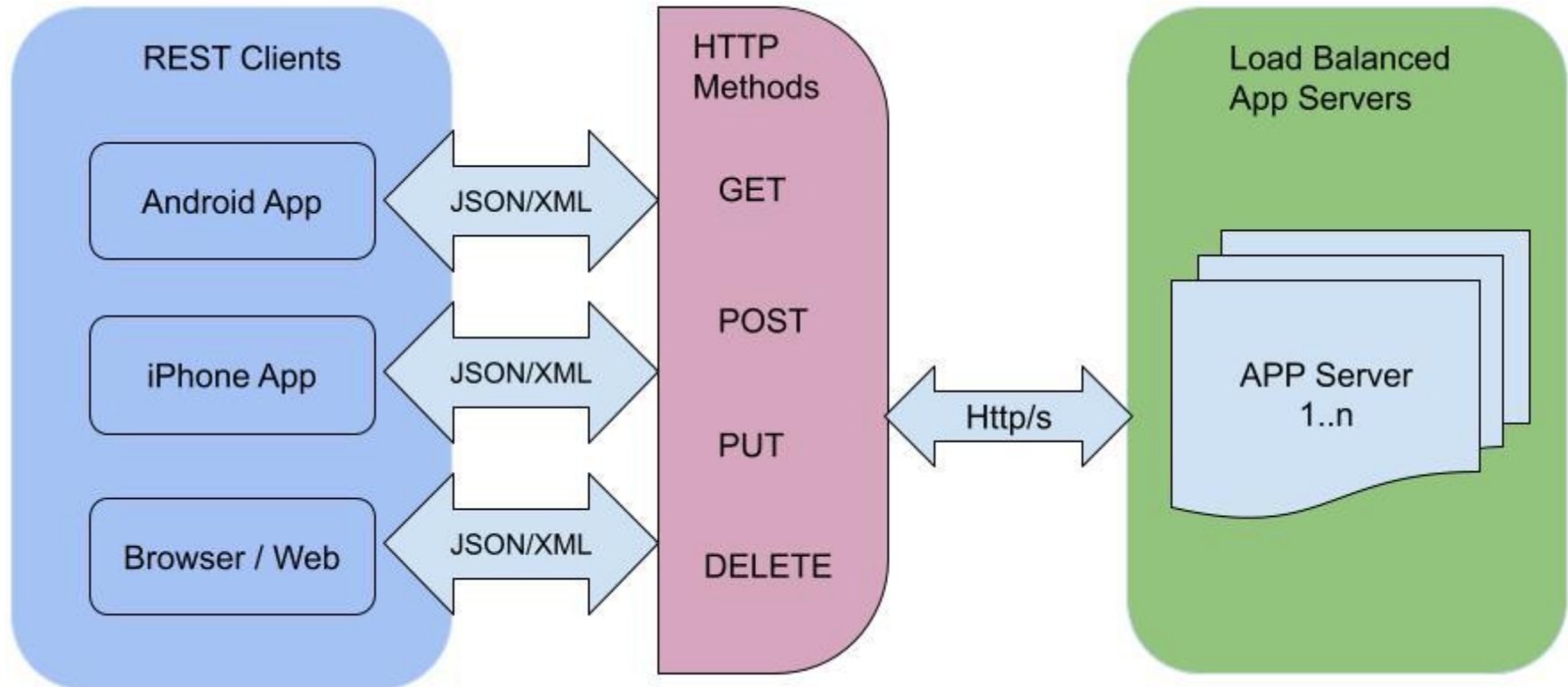
XML format

```
<user>
    <id>1</id>
    <name>Shreyas</name>
    <profession>Teacher</profession>
</user>
```

JSON format

```
{
  "id" : 1,
  "name" : "Shreyas",
  "profession" : "Teacher"
}
```

REST ARCHITECTURE



REST - Examples

HTTP Method	URI	Operation
GET	https://www.bits.edu/institute/students	For fetching information for all students
GET	https://www.bits.edu/institute/students/123	For fetching the student having ID 123
POST	https://www.bits.edu/institute/students	Submit the student information
PUT	https://www.bits.edu/institute/students/123	Update the student information for student with ID 123
DELETE	https://www.bits.edu/institute/students/123	Delete the student with ID 123

Sample curl commands

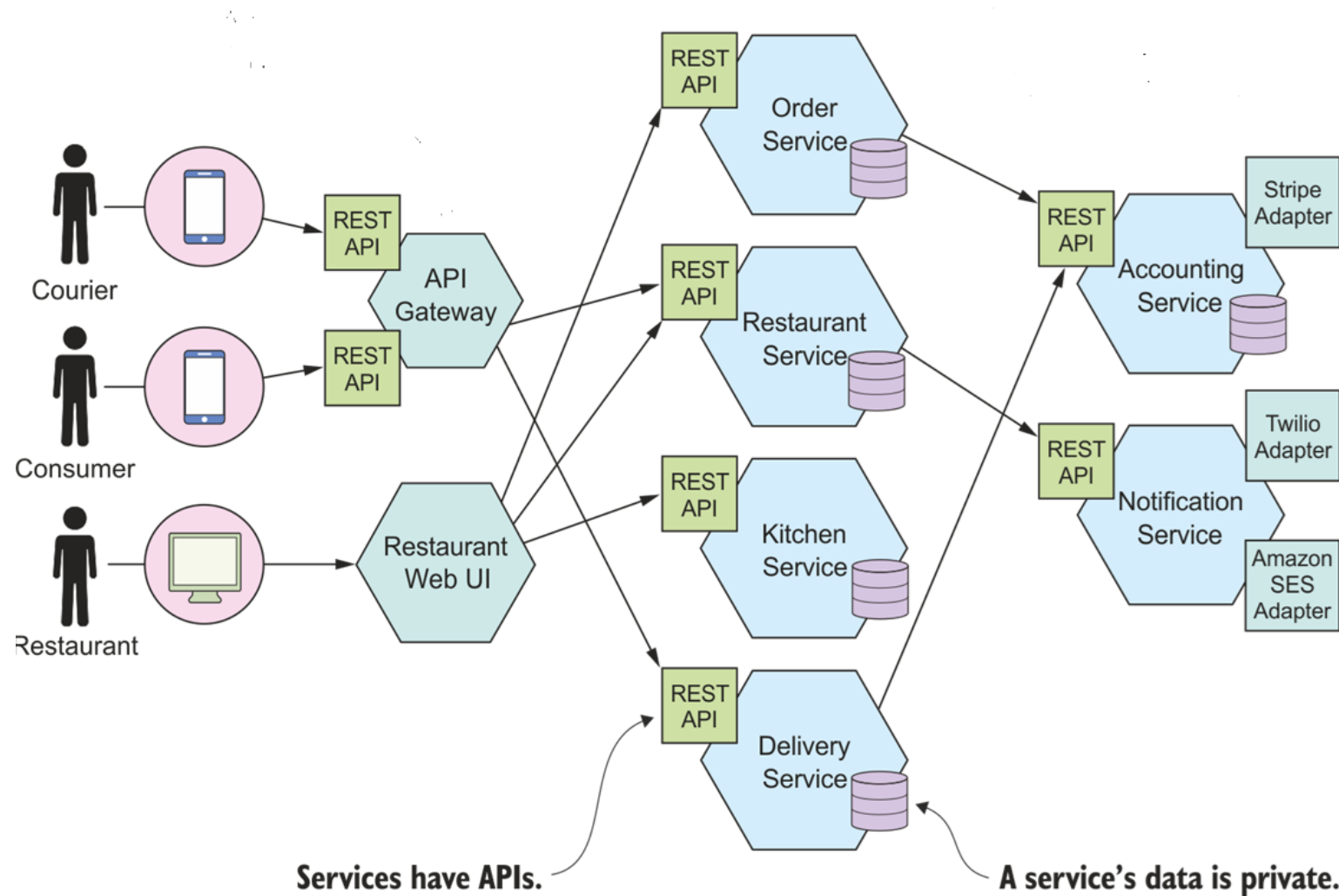
```
curl -X GET https://www.bits.edu/institute/students
```

```
curl -X GET "https://www.bits.edu/institute/students/123"
```

Guidelines for REST API design - <https://www.freecodecamp.org/news/rest-api-best-practices-rest-endpoint-design-examples/>

RESTful API based Application

[Food Delivery App]



REST – Benefits and Drawbacks

Benefits

- Mature and ubiquitous - de facto standard for APIs
- Testing a REST API is simple
- Supports synchronous request-response communication
- No intermediate broker
- Supported by most programming languages / frameworks

Drawbacks

- Reduced availability
- Fetching multiple resources [fetching a user's profile, their posts, and comments might require three separate API calls.]



Why RESTful APIs are important for this course?

Most AI services from AWS, Azure, or GCP support RESTful APIs

Ex: Amazon Bedrock is a tool to build generative AI apps with foundation models

Sample APIs from Amazon Bedrock

List models

Endpoint: <https://bedrock.us-west-2.amazonaws.com/models>

[Pass request with access tokens, and response in JSON]

Model Inference

Endpoint: <https://bedrock.us-west-2.amazonaws.com/models/{model-id}/invoke>

[Pass request with access tokens, and response in JSON]

* Same is true for HuggingFace, LangChain, Prefect which support RESTful APIs

Demo of the Bookstore application using RESTful APIs



BITS Pilani
Pilani Campus



GraphQL

Scenario - Fetch user profile, user posts and comments on the posts from a Social Media Application

REST Drawback - Fetching multiple resources requires multiple HTTP requests

REST

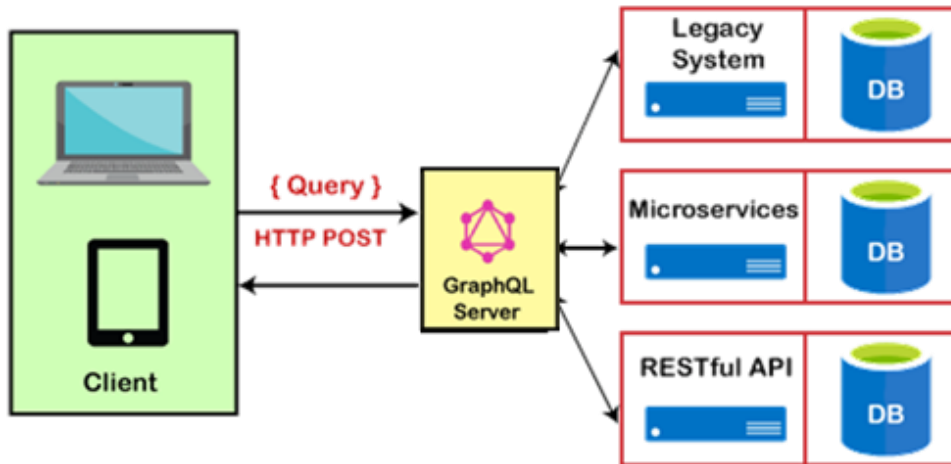
1. Fetch user profile - GET /users/{userId}
2. Fetch user posts - GET /users/{userId}/posts
3. Fetch comments on posts - GET /posts/{postId}/comments

GraphQL

- To overcome drawbacks of REST
- Facebook in 2015 came up with new API Syntax - GraphQL
- As an alternative to REST, **GraphQL is a versatile query language**, that lets developers construct requests that pull data from multiple data sources in a *single API call*. [Connect to multiple endpoints or databases]
- *Single request and all-inclusive reply*
- Uses 'query' operation to fetch data
- Uses 'mutation' operations to insert, update, or delete data.

<https://graphql.org/>

GraphQL



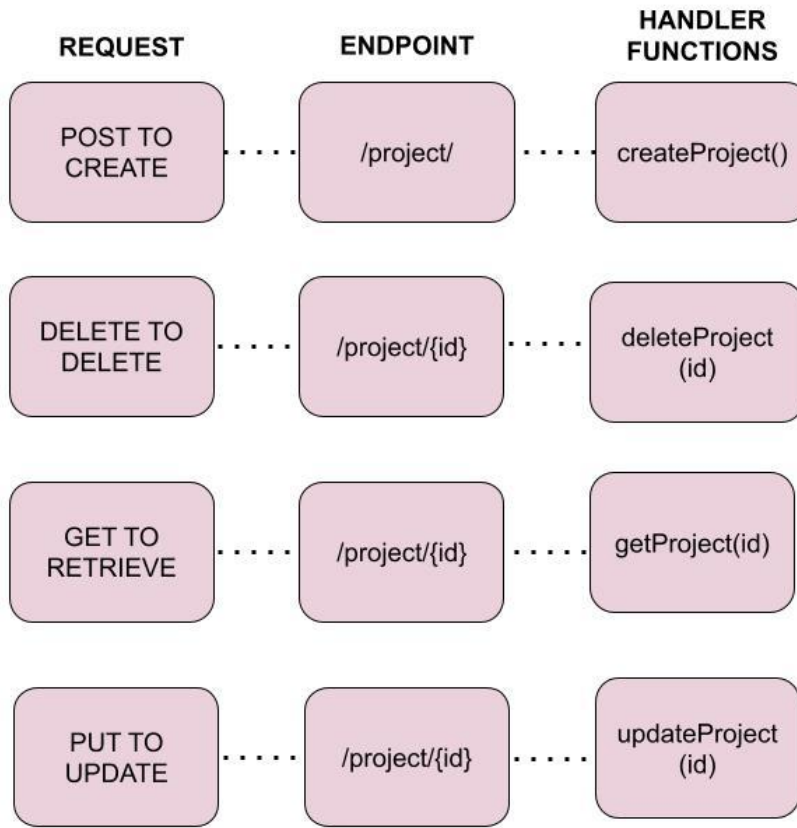
GraphQL Server receives client request (HTTP Post), and does following activities.

- 1. Map to Schema Definition:** Before it starts processing requests, the server defines a schema using GraphQL Schema Definition Language (SDL). This schema specifies the types of data that can be queried by the clients and the relationships between different data types. It acts as a blueprint that both the server and client understand.
- 2. Validation, Execution, and Response Formation:** When a request is made, the server first validates this request against the pre-defined schema to ensure it only asks for data that the schema supports. The server then executes this request on the **/graphql endpoint**, interacting with the necessary databases or data sources to gather the requested data. Finally, it forms a response in JSON format, which is sent back to the client.

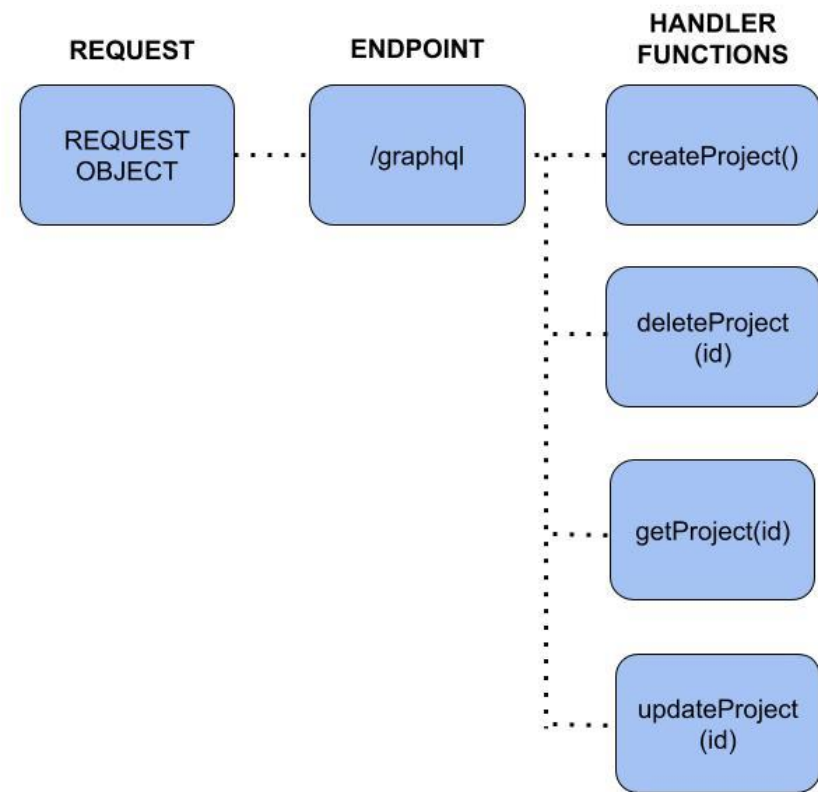
GraphQL



REST



GraphQL



GraphQL

GraphQL Query

```
query {  
  books {  
    title  
    author  
    publishedDate  
  }  
}
```



Sample Data Model for “book”

```
{  
  "id": integer,  
  "title": "string",  
  "author": "string",  
  "isbn": "string",  
  "publishedDate": "string",  
  "price": integer  
}
```

Data in JSON format

```
{  
  "data": {  
    "books": [  
      {  
        "title": "To Kill a Mockingbird",  
        "author": "Harper Lee",  
        "publishedDate": "July 11, 1960"  
      },  
      {  
        "title": "1984",  
        "author": "George Orwell",  
        "publishedDate": "June 8, 1949"  
      },  
      {  
        "title": "The Great Gatsby",  
        "author": "F. Scott Fitzgerald",  
        "publishedDate": "April 10, 1925"  
      }  
    ]  
  }  
}
```

REST vs GraphQL - Example

Scenario - Fetch user profile, user posts and comments on the posts from a Social Media Application

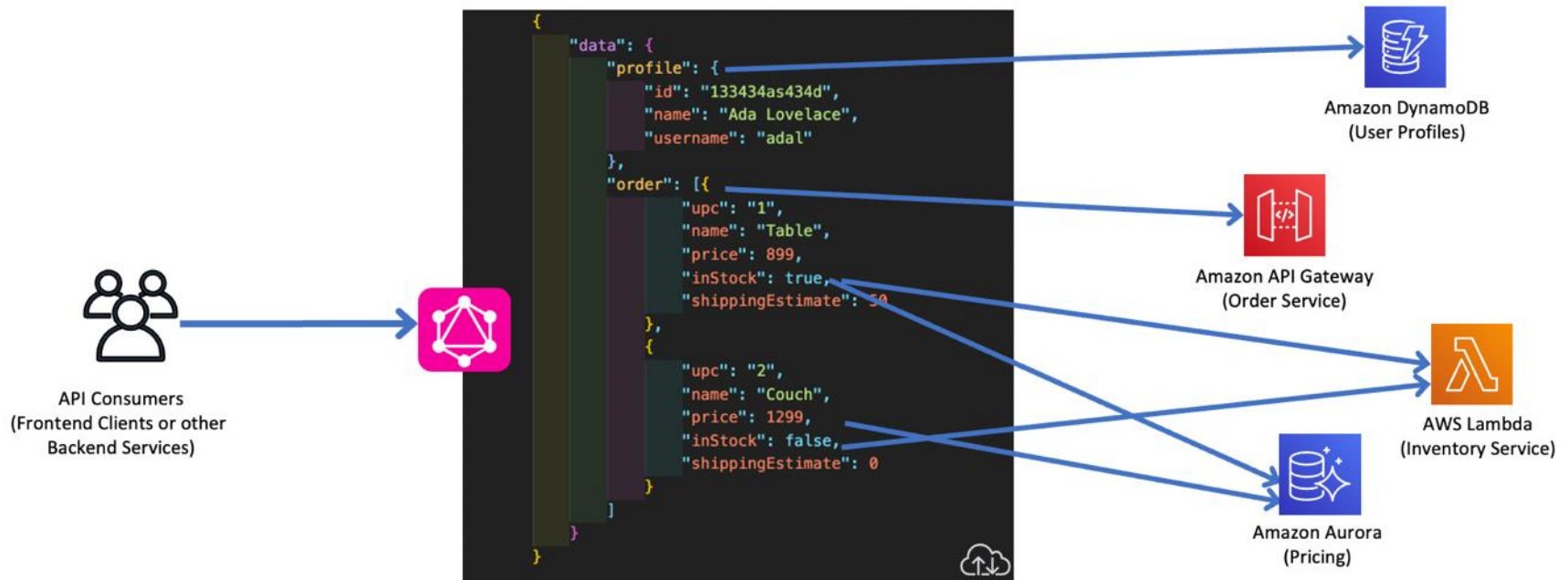
REST

1. Fetch user profile - GET /users/{userId}
2. Fetch user posts - GET /users/{userId}/posts
3. Fetch comments on posts - GET /posts/{postId}/comments

GraphQL

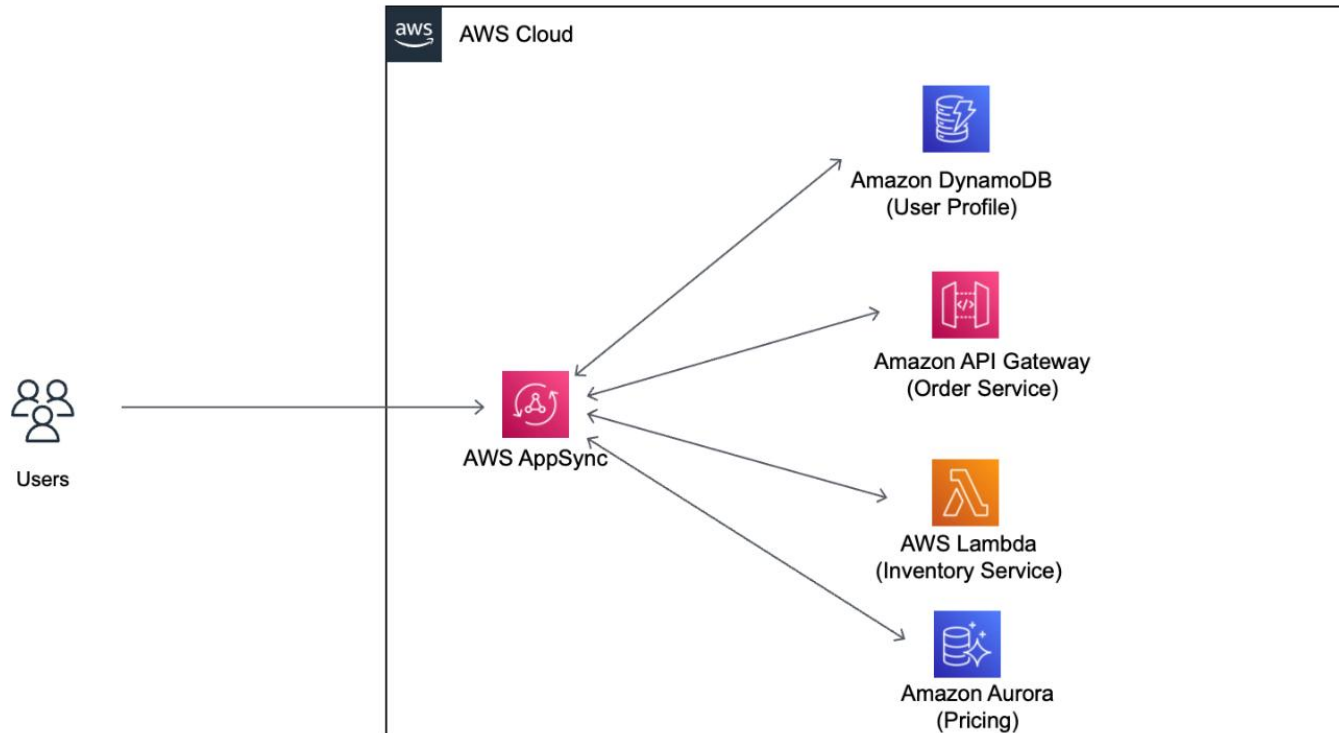
```
query {  
  user(id: "123") {  
    id  
    name  
    posts {  
      id  
      title  
      comments {  
        id  
        content  
      }  
    }  
  }  
}
```

Real Example - AWS



<https://aws.amazon.com/blogs/architecture/what-to-consider-when-modernizing-apis-with-graphql-on-aws/>

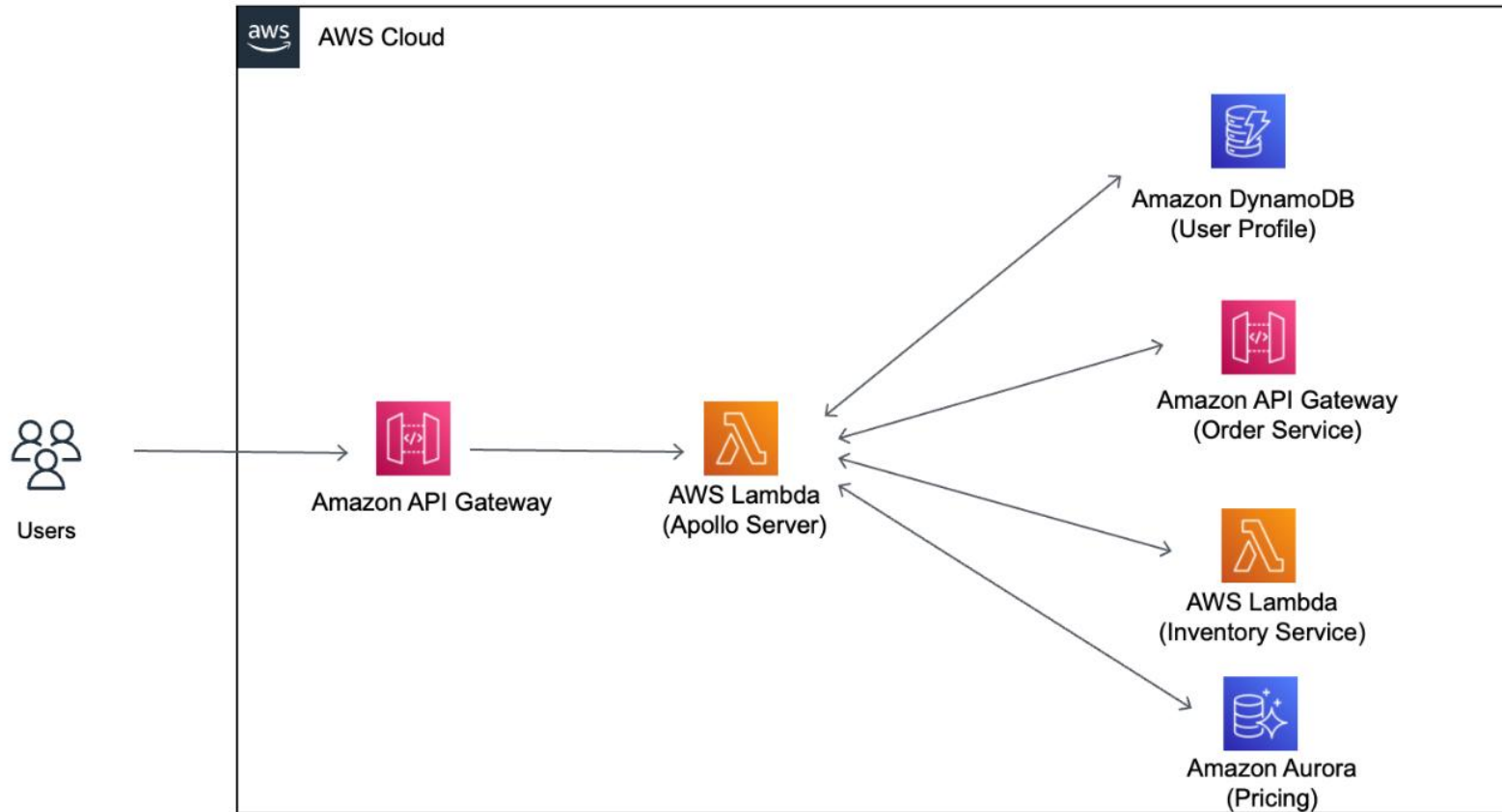
Option 1: Fully managed AWS Service (AppSync)



*AppSync is a managed GraphQL server that coordinates front end requests with backend services

<https://aws.amazon.com/blogs/architecture/what-to-consider-when-modernizing-apis-with-graphql-on-aws/>

Option 2: Self-Managed GraphQL



<https://aws.amazon.com/blogs/architecture/what-to-consider-when-modernizing-apis-with-graphql-on-aws/>

Demo - GraphQL

- Show a GraphQL Query Demo using Strawberry GraphQL Library in Python using the code first approach

Some terms in the code

`@strawberry.type`: A decorator used to define GraphQL types that map to Python classes.

`@strawberry.field` -> Method should be exposed as a field in the GraphQL schema. This means that the method can be queried directly by a GraphQL client.



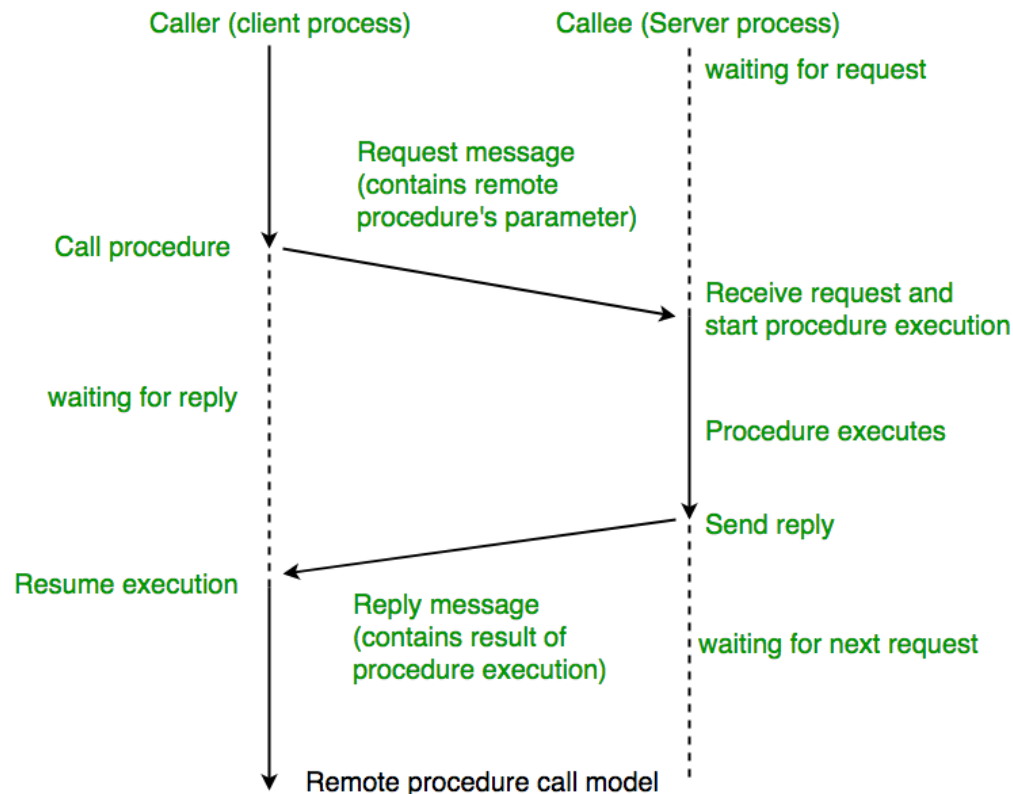
BITS Pilani
Pilani Campus



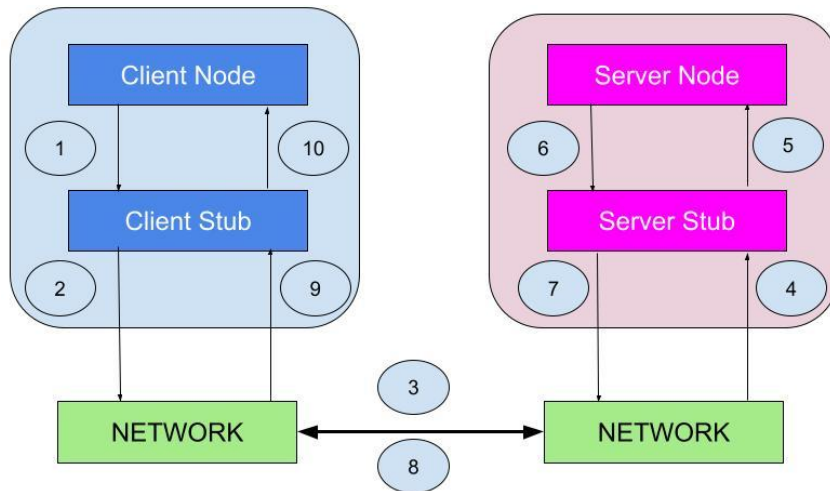
gRPC

RPC

- Remote Procedure Call (RPC) is a communication between client and remote server
- Used for distributed, client-server applications



RPC



Client Node and Client Stub: The client node makes a procedure call, which is handled by the client stub. The stub packages the call into a message and sends it over the network.

Network: The network facilitates the transmission of messages between the client and server stubs.

Server Stub and Server Node: The server stub receives the message, unpacks it, and passes the call to the server node. The server node processes the request and returns the result back through the server stub, which then sends it back over the network to the client stub.

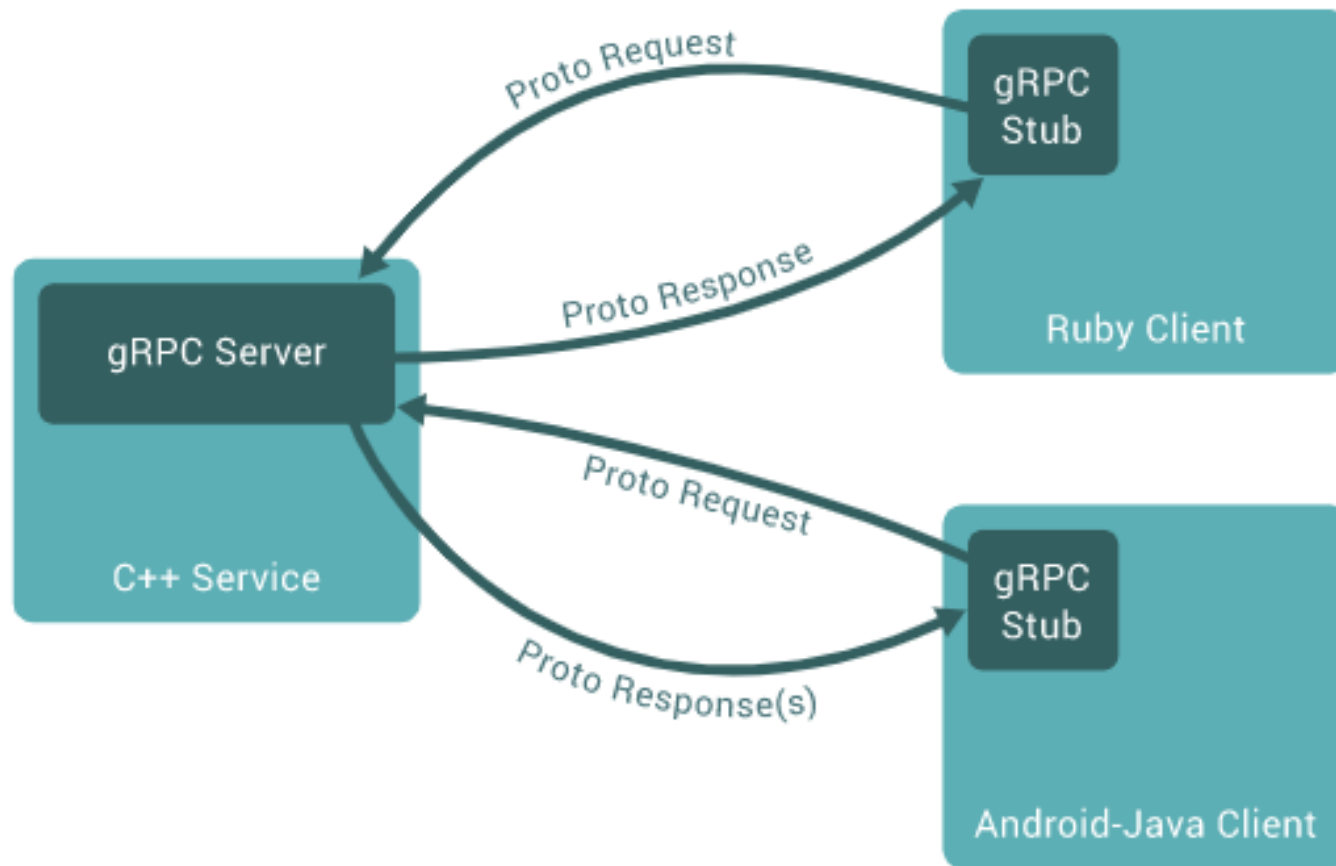
Client Node: Finally, the client stub receives the response, unpacks it, and delivers it to the client node.

gRPC



- Initially created by Google in 2015
- gRPC is a robust open-source RPC framework to build scalable APIs
- Protocol Buffers are used for data exchange, instead of JSON
- Uses HTTP/2 as the protocol [Instead of HTTP]
- 'Protoc' is the compiler
- Supports 10+ languages - C#.NET, C++, Dart, Go, Java, Kotlin, Node, Objective-C, PHP, Python, Ruby

gRPC



gRPC proto file [API definition]

```
// The greeter service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```

- Once we have defined gRPC proto file, we can use the “protobuf compiler - protoc” to generate code in our preferred programming language.
- This generated code includes the classes and methods needed to create and parse the messages defined in the proto file, as well as client and server stubs that can be used to call the methods defined in the file.
- By using gRPC proto, we can define our API in a language- and platform-neutral way, and then generate code for multiple programming languages.

gRPC proto file for Calculator

```
syntax = "proto3";

package calculator;

service Calculator {
  rpc Add (AddRequest) returns (AddResponse) {}
  rpc Multiply (MultiplyRequest) returns (MultiplyResponse) {}
}

message AddRequest {
  int a = 1;
  int b = 2;
}

message AddResponse {
  int result = 1;
}

message MultiplyRequest {
  double a = 1;
  double b = 2;
}

message MultipleResponse {
  double result = 1;
}
```

gRPC Demo

- Proto file -> “calculator.proto”
- Server logic implemented in Python listens on a port.
- Client logic implemented in Python sends requests to server, and gets response of the calculator functionality.

Ref URL - <https://grpc.io/docs/languages/python/quickstart/> for Python



gRPC Advantages and Disadvantages

Advantages

- It has simple, well-defined service interfaces and schema
- It is polyglot (supports multiple languages)
- Lightweight and Fast API
- Best for Inter-Service Communication

Disadvantages

- It may not be suitable for external-facing services
- The support for gRPC in browser and mobile applications is still in the primitive stages [grpc-Web extension is available for limited browsers]



Comparison

Comparison

Feature	Best API Type
Ubiquitous standard for Web	REST
Data Fetch	GraphQL
Browser Support	REST / GraphQL
Request Caching	REST
Code Generation	gRPC - Native support for 10+ languages GraphQL - GraphQL Code Generator [3 RD party] REST – Swagger [3 rd party]
Payload data structure	GraphQL - JSON REST - JSON & XML gRPC - Protocol Buffers



BITS Pilani
Pilani Campus

API Versioning

API Versioning

- API versioning is the practice of managing changes to an API and ensuring that these changes are made without disrupting clients.
- A good API versioning strategy clearly communicates the changes made and allows API consumers to decide when to upgrade to the latest version at their own pace.
- Why it is important?

When a third-party application developer uses your API to build an integration, they expect that the API will be stable. If you make changes to your API without considering clients, it forces them to change their own software. Otherwise, their applications could break if updates aren't accounted for.

<https://blog.hubspot.com/website/api-versioning#:~:text=What%20is%20API%20versioning%3F,version%20at%20their%20own%20pace.>



API Versioning

When to version API?

- Costly (Cost and Effort) for API Consumers and Developers
- Best to consider in case of *breaking change* - change in API that causes client applications to fail

Breaking changes are:

- Changing the format of request or response data (like switching from JSON format to XML format)
- Changing the data type of a resource (like changing from a string to an integer)
- Changing the name of a resource
- Removing one or more resources, removing or changing properties or methods for a particular resource, or any other changes to API functionality
- Adding a new required field for client HTTP requests.

<https://blog.hubspot.com/website/api-versioning#:~:text=What%20is%20API%20versioning%3F,version%20at%20their%20own%20page.>



API - Semantic Versioning

- Semantic Versioning is a **versioning scheme for using meaningful version numbers**.
- A normal version number **MUST** take the form X.Y.Z where X, Y, and Z are non-negative integers, and **MUST NOT** contain leading zeroes.
- **X is the major version**, **Y is the minor version**, and **Z is the patch version**. Each element **MUST** increase numerically.
- Ex: 1.9.0; 2.11.1

Google Maps JavaScript API - Current Version is 3.59.6 (23-January-2025)

URL - <https://developers.google.com/maps/documentation/javascript/releases#3.59.6>

API - Semantic Versioning

- **Major version:** In this approach, your URI would denote the *breaking changes to the API*. A new major version requires *creating a new API*. The version number is what you use to route to the correct host via your URI. Indicates incompatible API changes.
- **Minor version:** You update change logs to inform API consumers of *new functionality or bug fixes*. It introduces a *non-breaking functionality*.
- **Patch versions:** A patch is an update which is meant to *fix existing bugs* and it's backward compatible. It introduces *non-breaking functionality*.

Semantic Versioning - Example

Example - API Versioning for Movies

Version 1.0.0: This is the initial release of the API. It provides basic information about movies, such as the title, director, release year, and plot summary.

Version 1.1.0: This version adds some new features to the API. For example, it may allow users to search for movies based on specific criteria, such as genre or actor.

Version 1.1.1: This is a patch release that fixes some bugs in the 1.1.0 version of the API. It doesn't introduce any new features.

Version 2.0.0: This is a major release that introduces significant changes to the API. For example, it may switch to a new data format or add new endpoints for accessing movie data. The new version may not be backward compatible with the previous version, so clients will need to update their code to work with the new API.

Version 2.1.0: This version adds some new features to the 2.0.0 version of the API, but still maintains backward compatibility with the previous version.

Version 2.1.1: This is a patch release that fixes some bugs in the 2.1.0 version of the API. It doesn't introduce any new features

Semantic Versioning - Example

Each version of the API would be accessible through a specific endpoint, such as:

/api/v1.0.0/movies: Endpoint for accessing basic movie information in version 1.0.0 of the API.

/api/v1.1.0/movies/search: Endpoint for searching movies based on specific criteria in version 1.1.0 of the API.

/api/v2.0.0/movies: Endpoint for accessing movie data in version 2.0.0 of the API.

/api/v2.1.0/movies/search: Endpoint for searching movies based on specific criteria in version 2.1.0 of the API.



API Examples

[Self Study]

1. PetStore API

- Swagger Petstore - OpenAPI 3.0
- URL - <https://editor.swagger.io/>
- **Observe the JSON based API structure**

2. Rapid API

- World's largest public API Marketplace
- URL - <https://rapidapi.com/>
- Explore the APIs

3. Conference API

R2: “Mastering API Architecture - Design, Operate and Evolve API-Based Systems”, by J.Gough et al., O’Reilly, 2023

4. AsyncAPI Specification

URL - <https://www.asyncapi.com/en>



Thank You!